

Sistema de Detección Temprana de Somnolencia

Entregable E2: Diseño Detallado, Visión Artificial y Transmisión IoT

Arquitectura Híbrida: ESP32-CAM y Raspberry Pi 3 B+

UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS

Planteamiento del Problema

La somnolencia al volante es una de las causas principales de accidentes de tránsito. Según la OMS, la fatiga contribuye al **20% de los accidentes viales graves**.

En el Perú, esto impacta severamente en rutas interurbanas donde los conductores operan con déficits de sueño pronunciados.

El proyecto detecta de forma temprana los estados intermedios (como **micro-sueños de 4 a 6 segundos**), interviniendo antes de que ocurra una tragedia fatal.



Propuesta de **Solución Híbrida**



Visión Artificial

Uso de cámara y algoritmos mediante **OpenCV** para analizar cuadros continuos e identificar características biométricas asociadas a la fatiga en tiempo real.



Procesamiento Local

Una **Raspberry Pi 3 Model B+** procesa localmente con baja latencia, procesando las imágenes transmitidas por módulos como el ESP32-CAM.



Alertas y Red

Activación inmediata de alarmas físicas (LED, Buzzer) y transmisión de la notificación de riesgo mediante **Wi-Fi** hacia una plataforma IoT remota.

Parámetros de **Fatiga Visual**

-  **Tiempo de cierre de los ojos:** Se monitorea el Eye Aspect Ratio (EAR). Cierres prolongados indican fuerte probabilidad de micro-sueños.
-  **Frecuencia de parpadeo:** Alteraciones significativas en el patrón natural de parpadeo (demasiado lento o esporádico).
-  **Apertura de la boca:** Detección de bostezos mediante el cálculo de distancias faciales (Mouth Aspect Ratio).
-  **Inclinación de la cabeza:** Monitoreo del cabeceo y pérdida del tono muscular del cuello utilizando estimación de postura.

Arquitectura: Raspberry Pi 3 Model B+

COMPONENTE	ESPECIFICACIÓN TÉCNICA	ROL EN EL PROYECTO
Procesador	Broadcom BCM2837B0 (ARM Cortex-A53 64 bits)	Ejecución de SO Linux y algoritmos OpenCV
Memoria RAM	1 GB LPDDR2	Almacenamiento de buffers de video y variables
Conectividad	Wi-Fi (802.11 b/g/n/ac) + Bluetooth 4.2 BLE	Recepción de video y envío de notificaciones IoT
Interfaces	40 pines GPIO, puerto CSI	Control directo de LEDs, Buzzer y cámara local

Arquitectura: ESP32-CAM

Nodo Remoto de Adquisición

Módulo que integra el SoC ESP32 con núcleo Xtensa LX6 (32 bits, 240 MHz). Opera mediante firmware bare-metal o RTOS.

En este proyecto, se transforma en un **servidor de transmisión de video inalámbrico**.

Especificaciones Clave

Memoria: 520 KB SRAM interna (soporta memoria PSRAM externa para buffers de video de mayor resolución).

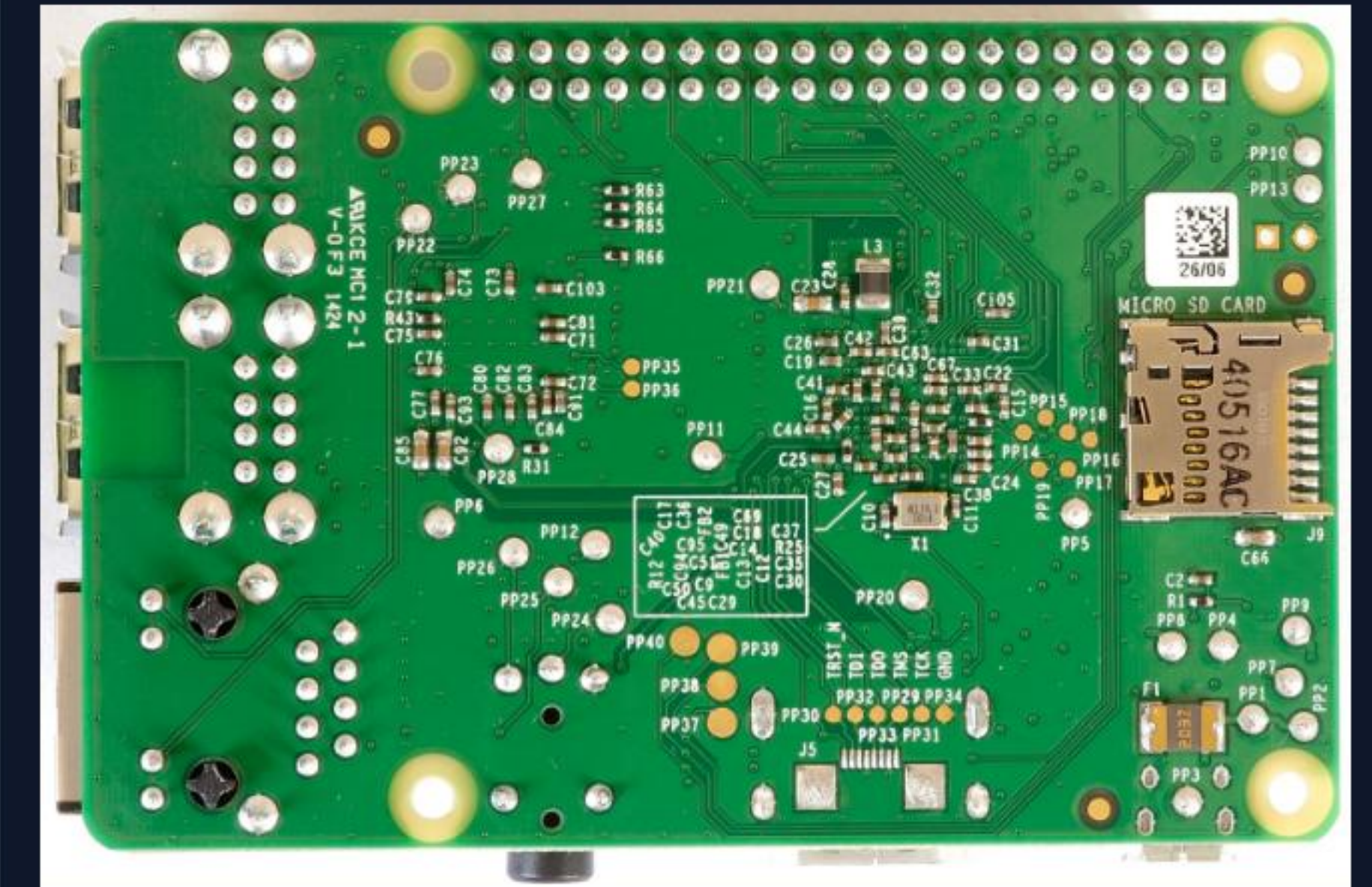
Cámara: Sensor OV2640 o AI_Thinker.

Conectividad: Wi-Fi y Bluetooth nativos.

Organización de Memoria (SBC)

La Raspberry Pi gestiona la memoria de manera unificada mediante el sistema operativo Linux (Raspberry Pi OS).

-  El **1GB RAM** se comparte dinámicamente entre la CPU y la GPU.
-  El procesamiento multicore permite manejar el flujo de video (cámara CSI o HTTP Stream) sin bloquear las tareas de red o GPIO.
-  La interfaz CSI accede directamente a la memoria RAM vía DMA (Direct Memory Access).



Gestión a Nivel **Firmware**



Estructura de Memoria

El código gestiona datos temporalmente en **SRAM** (lecturas de sensores, buffers de comunicación UART). Las constantes y algoritmos residen en **Flash**.



Control de Registros

A nivel de hardware (como en arquitecturas AVR), se controlan periféricos escribiendo bits en registros específicos (ej. **DDRD** para dirección, **PORTD** para salida alta/baja, **PIND** para lectura).



Uso de Buffers

Las transmisiones seriales (UART) no detienen la CPU; los datos se acumulan en un buffer SRAM, enviando bytes secuencialmente. Esto evita pérdida de información.

Interfaces de Comunicación

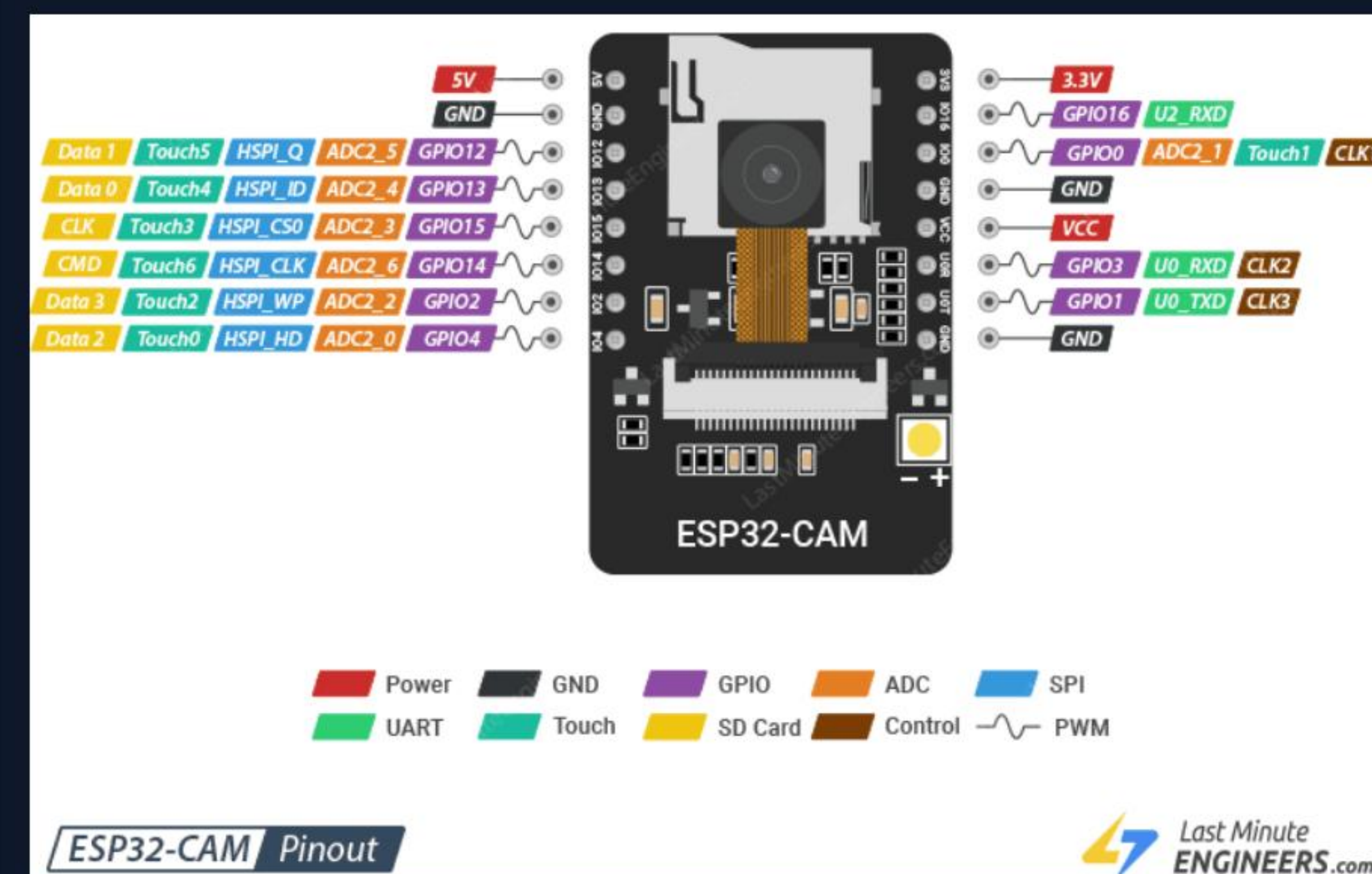
INTERFAZ	TIPO	APLICACIÓN EN EL SISTEMA
GPIO	Entrada/Salida Digital	Control directo del Buzzer (PWM) y LEDs indicadores.
UART	Serial Asíncrono	Transmisión serial entre Raspberry Pi y ESP32 o módulos BT.
I2C	Serial Síncrono	Expansión futura para sensores periféricos (SDA/SCL).
Wi-Fi	Red Inalámbrica	Streaming de video del ESP32 a la RPi y alertas a la nube.

Mapa de Pines: **ESP32-CAM**

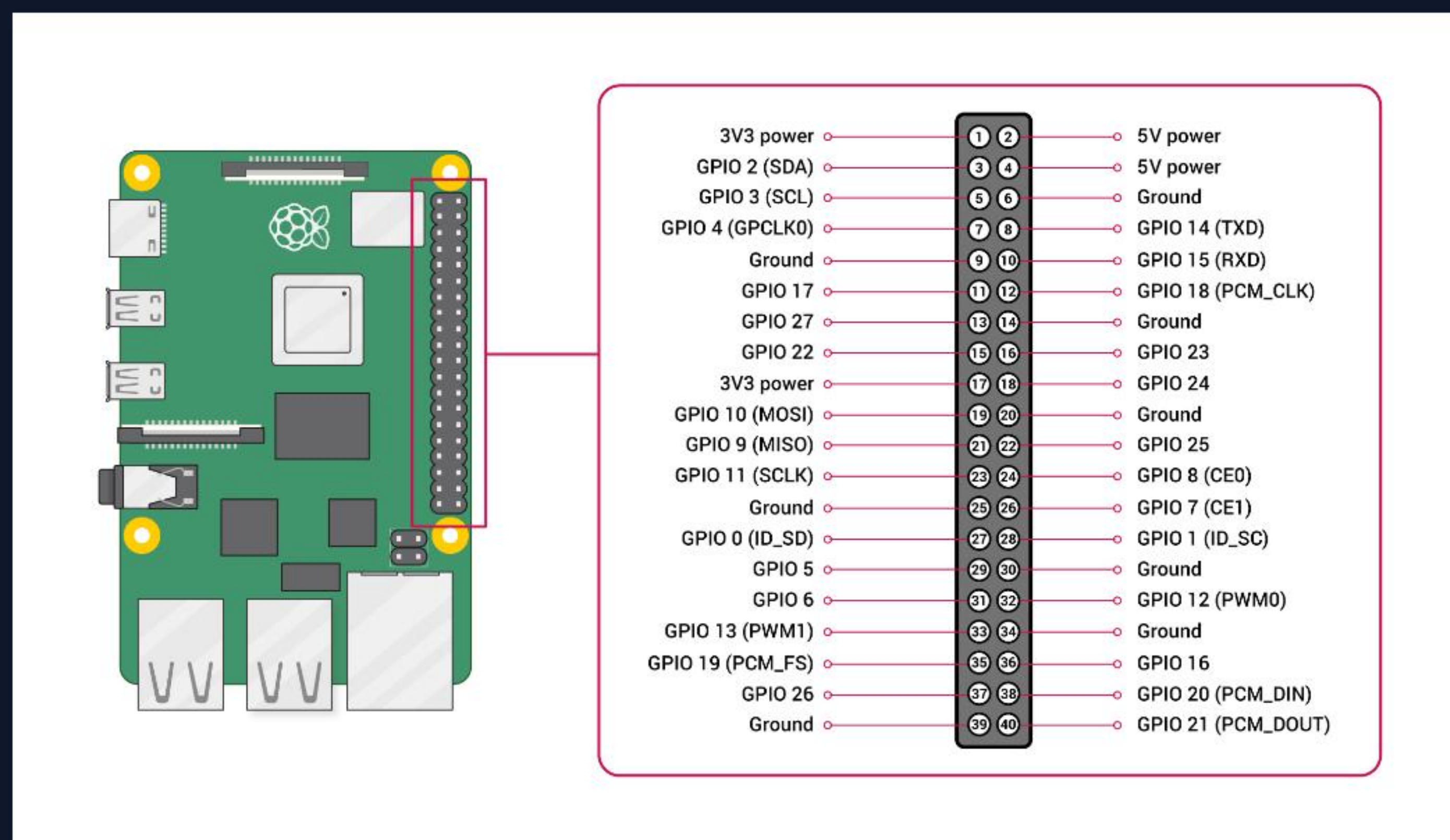
El ESP32-CAM reserva la mayoría de sus pines internos para la cámara y la memoria PSRAM/SD.

Para programación e interconexión, los pines expuestos son críticos:

-  **GPIO 1 y 3:** UART (U0TXD, U0RXD) usados para programar con módulo FTDI.
-  **MicroSD:** Comparte los pines GPIO 2, 4, 12, 13, 14, 15.
-  **GPIO 0:** Pin de arranque (debe estar a GND para flashear el código).



Mapa de Pines: Raspberry Pi



Distribución de actuadores conectados directamente al cabezal GPIO de 40 pines:

-  **GPIO 18:** Buzzer Piezoeléctrico (Alarma Sonora PWM).
-  **GPIO 23:** LED Rojo (Indicador de Somnolencia).
-  **GPIO 24:** LED Verde (Funcionamiento Normal).
-  **GPIO 25:** Pulsador Reset (Reinicio Manual).

Máquina de Estados Finitos (FSM)

Conjunto de Estados (S)

El sistema se rige por un flujo lógico estructurado:

S0: MONITOREO_NORMAL

S1: ADVERTENCIA

S2: ALARMA

S3: RECUPERACIÓN

Variables (I/O)

Entradas: O (Ojos cerrados), B (Bostezo), N (Notificación exitosa), T (Temporizador), D (Sin fatiga).

Salida: A (Activación de alarma local; A=1 Activa, A=0 Desactiva).

Tabla de Transiciones

ESTADO ACTUAL	CONDICIÓN EVALUADA	ESTADO SIGUIENTE	ACCIÓN DE SALIDA
MONITOREO	somnolencia $\geq$ 40% OR t_inactivo $\geq$ 3s	ADVERTENCIA	LED=ON, Buzzer breve
MONITOREO	somnolencia $\geq$ 60% OR t_inactivo $\geq$ 6s	ALARMA	LED parpadea, Buzzer continuo
ADVERTENCIA	somnolencia $<$ 15% AND t_inactivo $<$ 3s	RECUPERACIÓN	LED=OFF, Buzzer=OFF
ALARMA	Movimiento/Actividad detectada	RECUPERACIÓN	Cese de alertas

Flujo Operacional

El ciclo lógico del sistema se mueve de forma ininterrumpida a través de las siguientes fases secuenciales:

1. Adquisición

ESP32-CAM captura imágenes y las sirve como Stream de Video sobre IP.



2. Procesamiento

La Raspberry Pi con OpenCV calcula la fatiga basándose en los "landmarks" faciales.



3. Decisión FSM

La lógica clasifica el estado (Normal, Advertencia, Alarma) basado en umbrales.



4. Reacción

Activación de GPIO locales (LED/Buzzer) y alertas remotas vía Wi-Fi.

Implementación: Transmisión de Datos

Se configura la Raspberry Pi para procesar el video de red en Python.

```
# URL del Stream HTTP generado por ESP32-CAM
ip_addr = '192.168.2.29'
stream_url = 'http://' + ip_addr + ':81/stream'

# Procesamiento continuo de los chunks de red
res = requests.get(stream_url, stream=True)
for chunk in res.iter_content(chunk_size=100000):
    img_data = BytesIO(chunk)
    cv_img = cv2.imdecode(np.frombuffer(img_data.read(),
np.uint8), 1)
# Aplicar inferencia OpenCV / TensorFlow Lite
```

Ventaja Clave:

Al delegar la captura a un nodo ESP32, la Raspberry Pi dedica 100% de su CPU ARM Cortex-A53 al cálculo biométrico intensivo (Haar Cascades / EAR), reduciendo la latencia de respuesta general del sistema.

Continuidad: **Expansión IoT**

1. Telemetría a la Nube:

Implementar el protocolo **MQTT** (v3.1.1) para enviar datos en tiempo real (estado, alarmas) hacia un Broker (ej. Mosquitto, AWS IoT).

2. Visualización:

Construir paneles de control (Dashboards en Grafana o Node-RED) para supervisores de flota.

3. Inteligencia Computacional:

Migrar de umbrales estáticos a un modelo de Machine Learning adaptativo utilizando **TensorFlow Lite**.

Fleet Management



Conclusiones del Proyecto



Arquitectura Funcional

La combinación de ESP32-CAM (captura) y Raspberry Pi (procesamiento) demuestra ser una arquitectura distribuida óptima y de muy bajo costo.



Lógica Robusta

La Máquina de Estados Finitos (FSM) aísla correctamente los estados del conductor, previniendo falsas alarmas y escalando las alertas adecuadamente.



Seguridad Vial

Esta integración de tecnologías embebidas, visión artificial y telemetría IoT proporciona una solución real y escalable para evitar accidentes por fatiga en rutas.

¿Preguntas?

Gracias por su atención a la presentación técnica.

Arquitectura del Computador — Entregable E2

Image Sources



<https://thumbs.dreamstime.com/b/conductor-de-cami%C3%B3n-agotado-durmiendo-sobre-el-volante-cansado-en-la-direcci%C3%B3n-del-veh%C3%ADculo-dentro-cabina-camiones-233219622.jpg>

Source: es.dreamstime.com



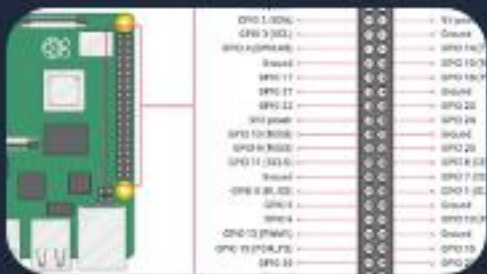
https://raspi.tv/wp-content/uploads/2014/07/Raspberry-Pi-B+REAR-cropped_1500.jpg

Source: raspi.tv



Thumbnail for <https://lastminuteengineers.com/wp-content/uploads/iot/ESP32-CAM-Pinout.png>

Source: lastminuteengineers.com



<https://lab.arts.ac.uk/uploads/images/gallery/2024-02/5loOy9yuofsJCrSk-image-1708087453528.png>

Source: lab.arts.ac.uk



<https://img1.bevywise.com/images/mqtt-dashboard2.webp>

Source: www.bevywise.com